

MapReduce

COMPCSI 752: Big Data Management

University of Auckland

Credits to Ninh Pham for this Lecture

Slides are collected and edited from <http://webdam.inria.fr/Jorge/index9213.html>
and <https://homepages.cwi.nl/~boncz/bads/mapreduce.shtml>

Auckland, May 2025

Outline

- The MapReduce framework
 - MapReduce
 - HDFS
 - Apache Hadoop
 - Apache Spark
- MapReduce algorithms
 - PageRank
 - Similarity join

Outline

- The MapReduce framework
 - MapReduce
 - HDFS
 - Apache Hadoop
 - Apache Spark
- MapReduce algorithms
 - PageRank
 - Similarity join

Big data analytics

数据治理

- The Moore's law of big data:

“We have generated more than 90% of data in the last two years itself.”

- Big data analytics:

“How to turn big data into value?”

“How to process this data?”



Unstructured

2016 What happens in an INTERNET MINUTE?



2017 This Is What Happens In An Internet Minute



2018 This Is What Happens In An Internet Minute



2019 This Is What Happens In An Internet Minute



2020 This Is What Happens In An Internet Minute

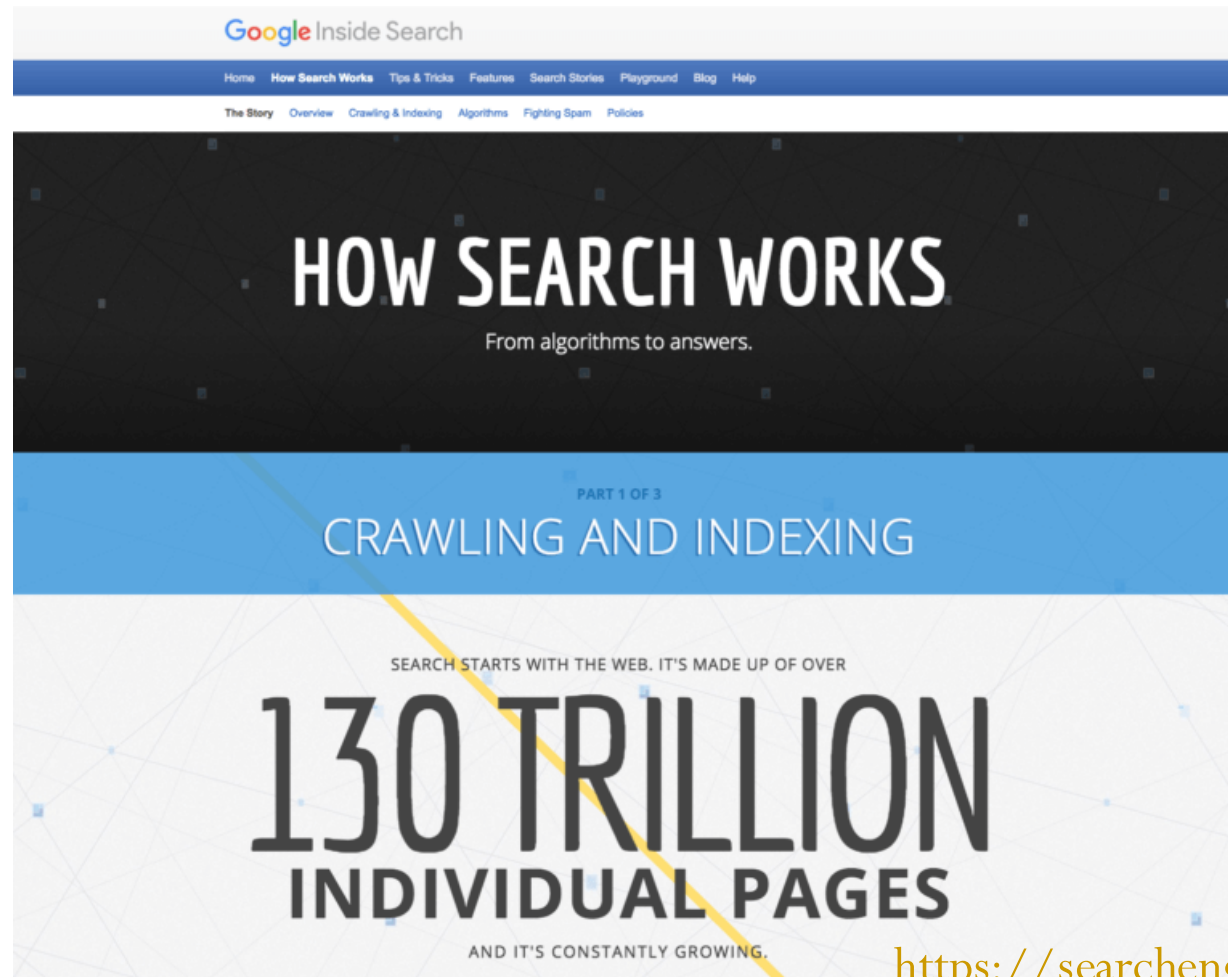


2021 This Is What Happens In An Internet Minute



Data analysis at a large scale

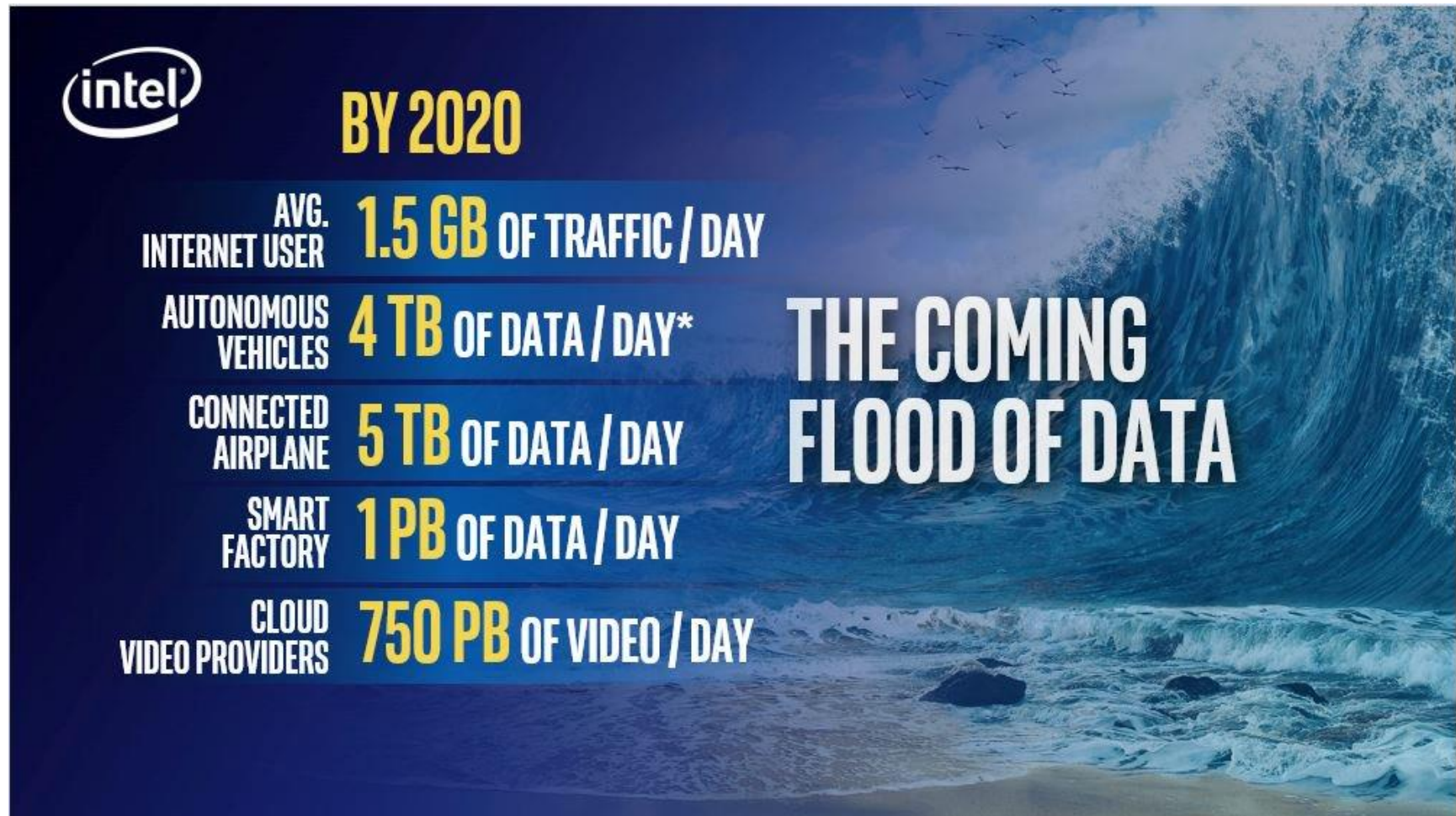
- **Big** data collections (TB to PB) stored on distributed file systems: search engine indexes, sensor data...



<https://searchengineland.com/>

Data analysis at a large scale

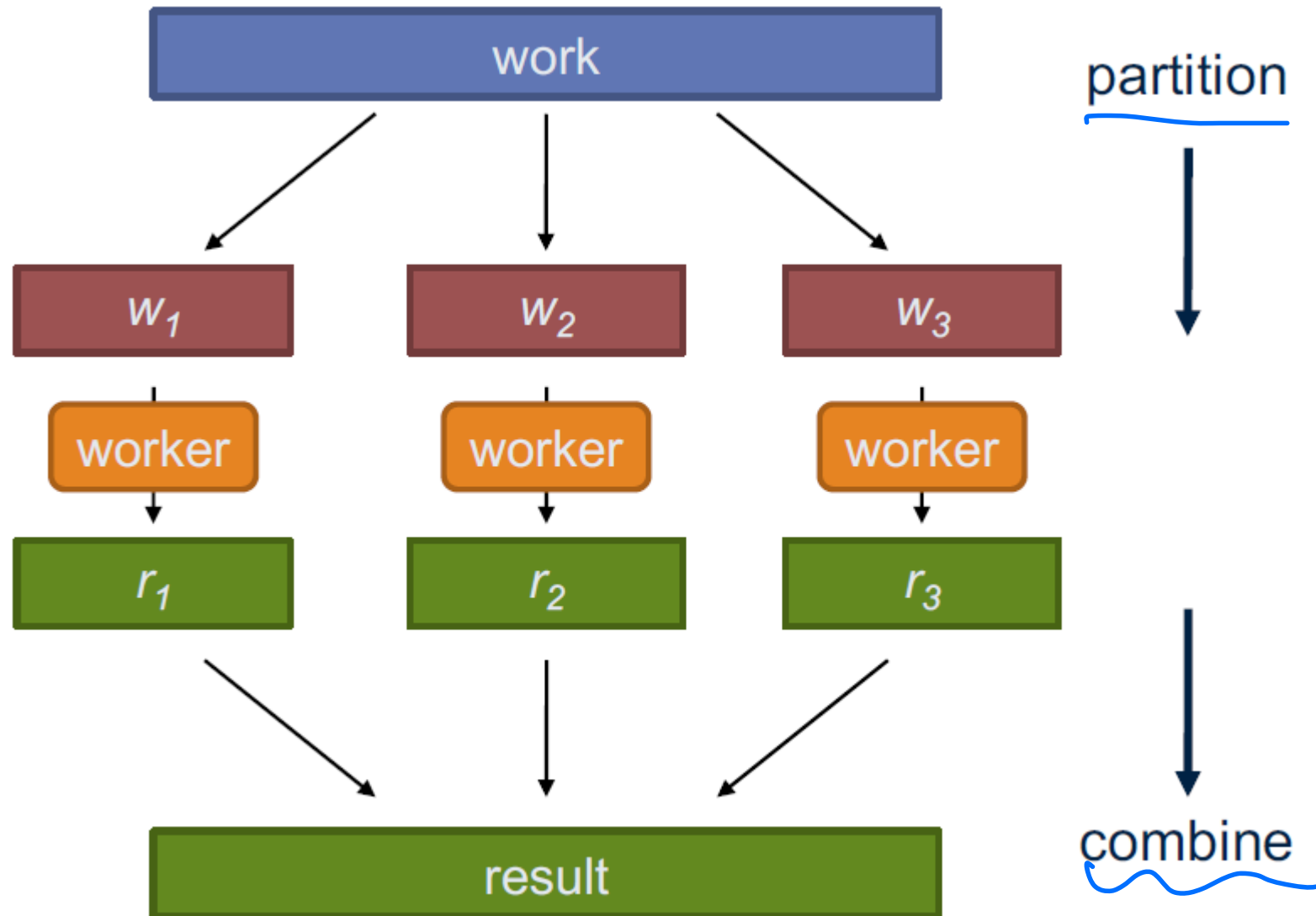
- Big data collections (TB to PB) stored on distributed file systems: search engine indexes, sensor data...



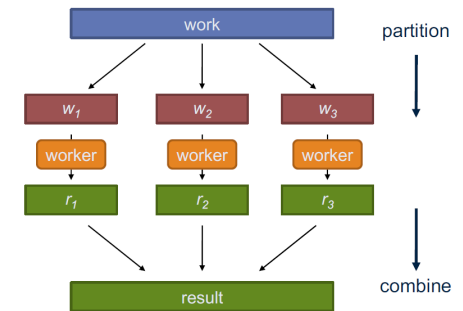
Data analysis at a large scale

- Big data collections (TB to PB) stored on distributed file systems: search engine indexes, sensor data...
 - Need efficient ways for reformatting, processing, analyzing...
 - Parallelization of computation, that benefits from the processing power of all nodes in a cluster
 - Resilience to failure
- Handwritten notes:*
- Above "efficient ways": 并行 (parallel)
- Above "reformatting": 系统 (system)
- An arrow points from "系统" to "Resilience to failure".
- Next to "Resilience to failure": 韧性 (resilience)

Key premise: Divide and conquer ^{分割}



Parallelization challenges



1. How to assign work units to workers? *用多长时间*
2. What if we have more work units than workers? *queue, 排队*
3. What if workers need to share partial results?
4. How to know all workers have finished? *monitoring, coordination*
5. What if workers die? *fault tolerance.*
6. What if data get lost while transmitted over the network
redundant storage.

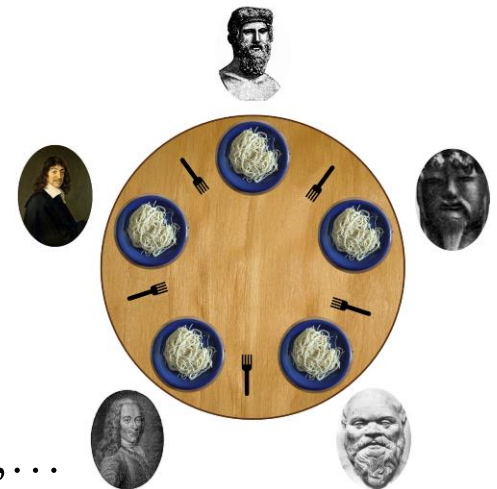
What is the common theme of all of these problems?

Common theme?

- Parallelization problems arise from:
 - Communication between workers (e.g. to exchange state)
 - Access to shared resources (e.g. data)
- Thus, we need a synchronization mechanism
同步机制

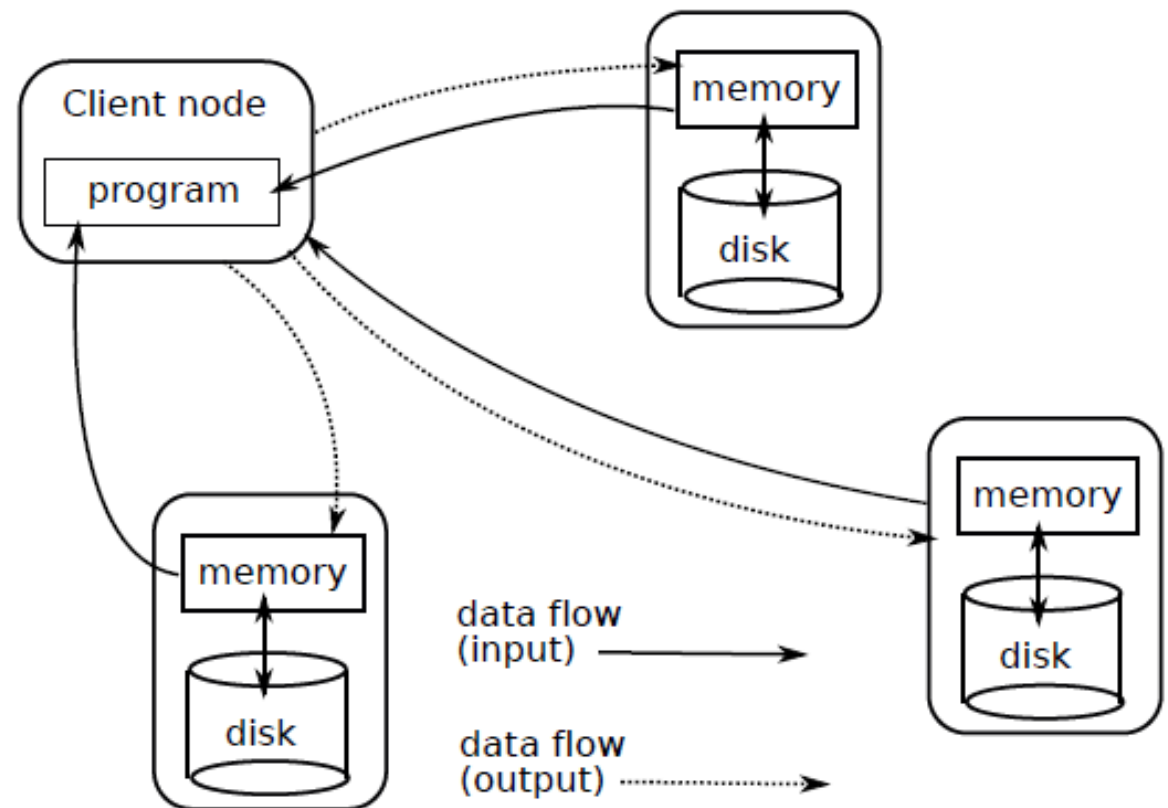
Managing multiple workers

- Difficult because
 - We do not know the order in which workers run
 - We do not know when workers interrupt each other
 - We do not know when workers need to communicate partial results
 - We do not know the order in which workers access shared data
- Thus we need:
 - Semaphores (lock, unlock)
 - Conditional variables (wait, notify, broadcast)
 - Barriers
- Still, more problems to handle
 - Deadlock, livelock, race conditions,...
 - Dining philosophers, sleeping barbers, cigarette smokers,...



Centralized computing with distributed data storage

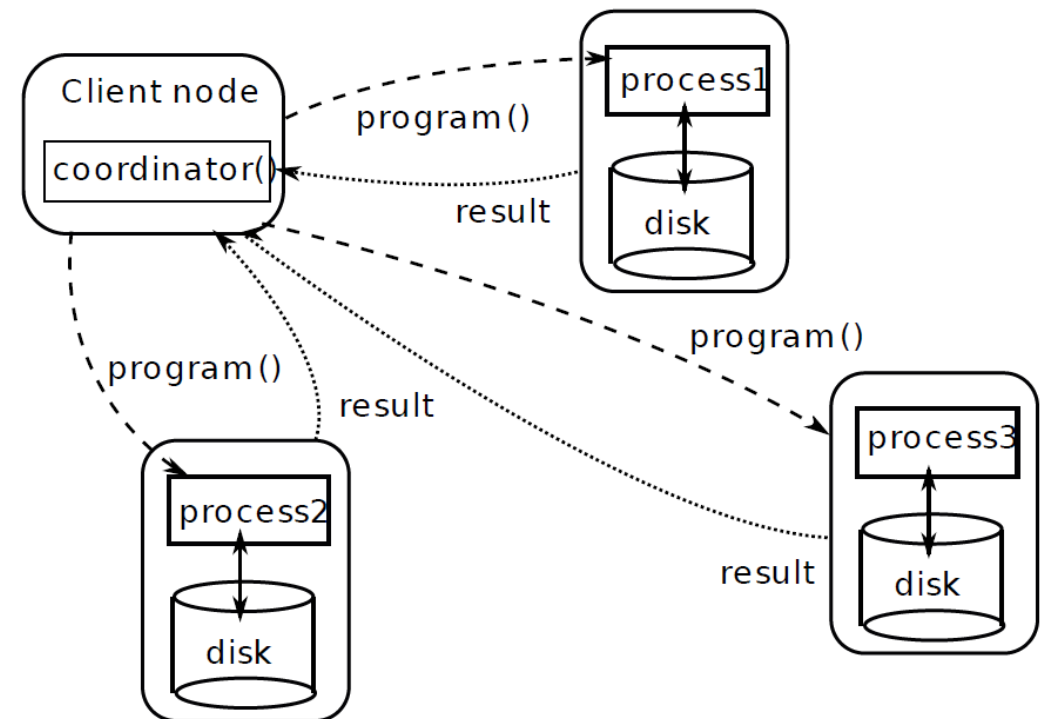
- Run the program at client node, get data from the distributed system
- **Downsides:** important data flows, no use of the cluster computing resources



Pushing the program near the data

避免传输大数据

- MapReduce: A programming model (inspired by standard functional programming operators) to facilitate the development and execution of distributed tasks
- De facto big data processing, open-source implementation in Hadoop (contributed by Yahoo)



MapReduce in brief

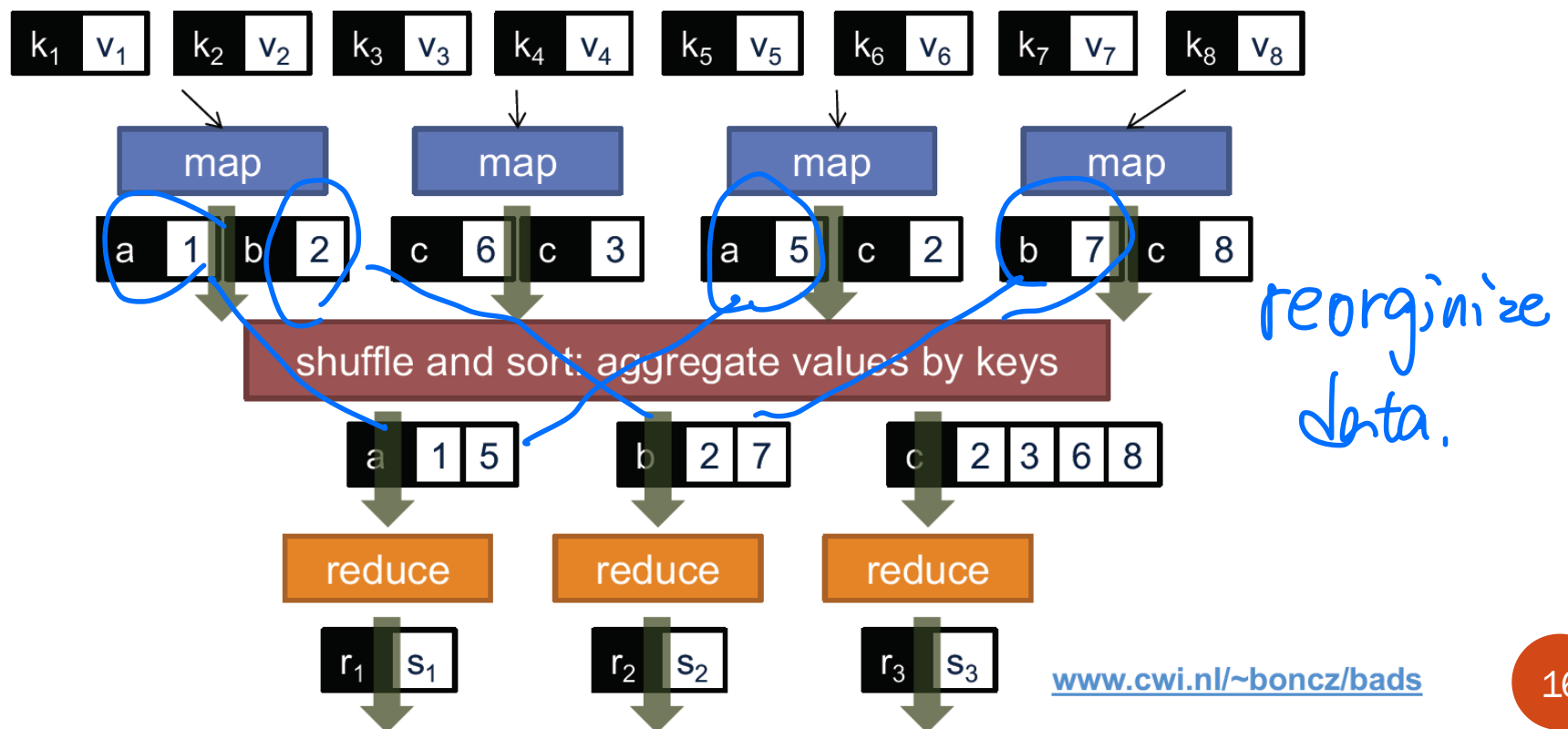
- The programmer defines the program logic as **two functions**.
 - *map*: transforms the input into key-value pairs to process *process input*
 - *reduce*: aggregates the list of values for each key *merge ----- 操作*
- The MapReduce environment takes charge of **distribution aspects** *改进 数据分布.*
- A complex program can be decomposed as a **succession** of Map and Reduce tasks

MapReduce in brief

map: $(k, v) \rightarrow [\langle k', v' \rangle]$

reduce: $(k', [v']) \rightarrow [\langle k'', v'' \rangle]$

- All values with the same key are sent to the same reducer



Three operations on key-value pairs

- User-defined: *map*: $(k, v) \rightarrow \text{list}(\langle k', v' \rangle)$ 用

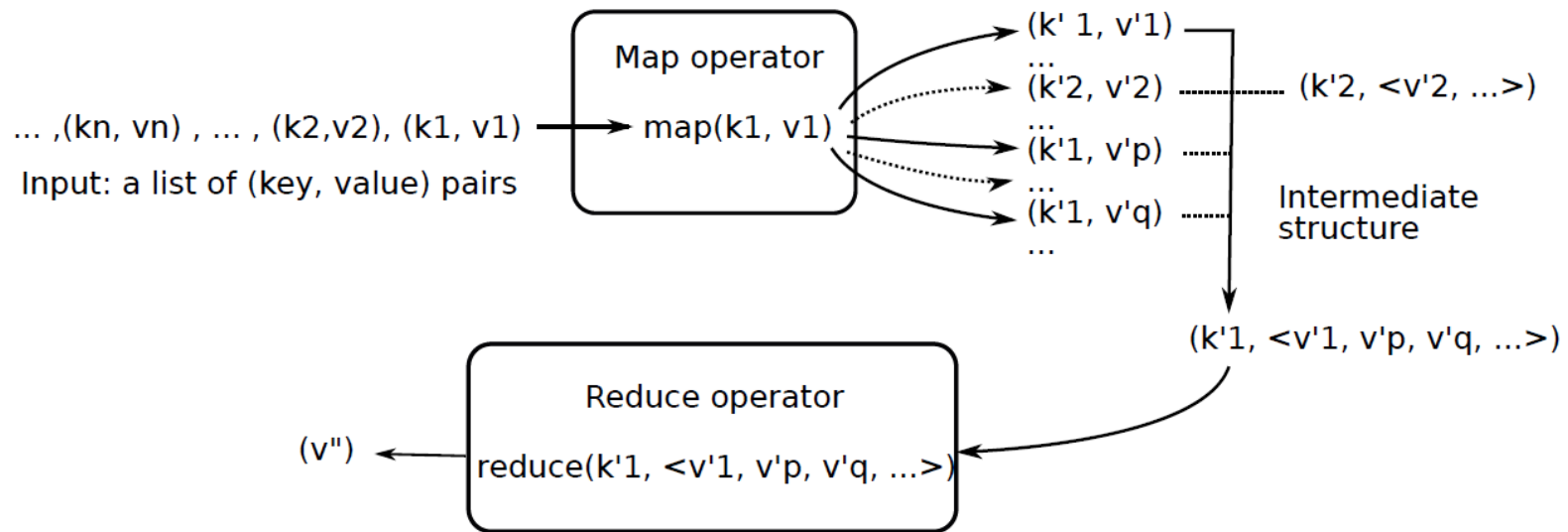
```
Map(String docid, String text):  
  for each word w in text:  
    Emit(w, 1);
```

- Fixed behavior: *shuffle*: $\text{list}(\langle k', v' \rangle) \rightarrow \text{list}(k', \text{list}(v'))$ 重新分组
regroups all intermediate pairs on the key
- User-defined: *reduce*: $(k', \text{list}(v')) \rightarrow \text{list}(\langle k'', v'' \rangle)$ 用

```
Reduce(String term, Iterator<Int> values):  
  int sum = 0;  
  for each v in values:  
    sum += v;  
  Emit(term, sum);
```

Job workflow in MapReduce

- Important: each pair, at each phase, is processed independently from the other pairs



- Network and distribution are transparently managed by the MapReduce environment

“Hello world”: Word count

```
Map(String docid, String text):  
    for each word w in text:  
        Emit(w, 1);
```

```
Reduce(String term, Iterator<Int> values):  
    int sum = 0;  
    for each v in values:  
        sum += v;  
    Emit(term, sum);
```

Example: Term count in MapReduce

URL	Document
u_1	the jaguar is a new world mammal of the felidae family.
u_2	for jaguar, atari was keen to use a 68k family device.
u_3	mac os x jaguar is available at a price of us \$199 for apple's new "family pack".
u_4	one such ruling family to incorporate the jaguar into their name is jaguar paw.
u_5	it is a big cat.

Example: Term count in MapReduce

term	count
jaguar	1
mammal	1
family	1
jaguar	1
available	1
jaguar	1
family	1
family	1
jaguar	1
...	

shuffle input

map output

term	count
jaguar	1,1,1, 1,1
mammal	1
family	1,1,1
available	1
...	

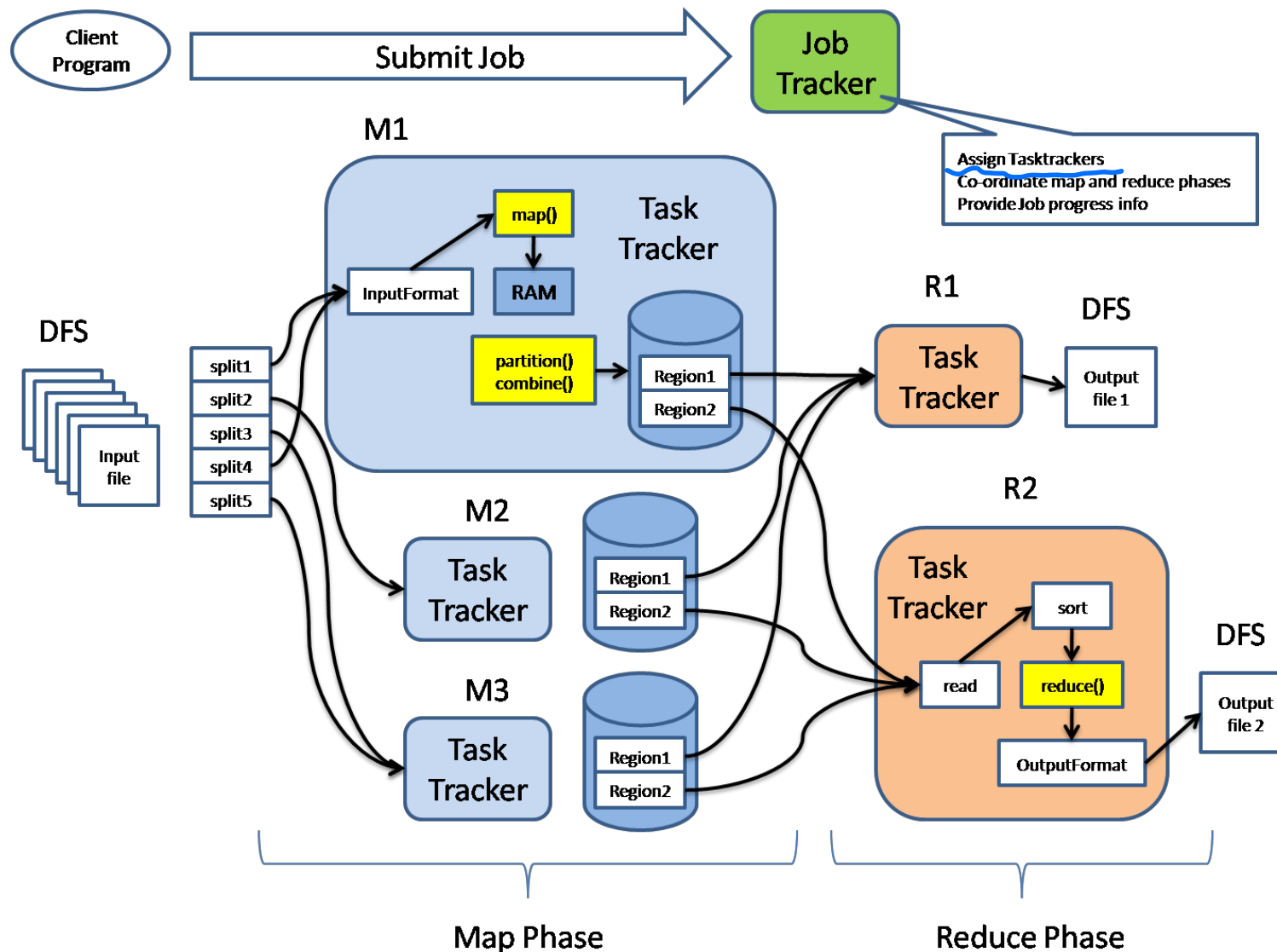
output
reduce input

shuffle

term	count
jaguar	5
mammal	1
family	3
available	1
...	

final output

Distributed execution of a MapReduce job



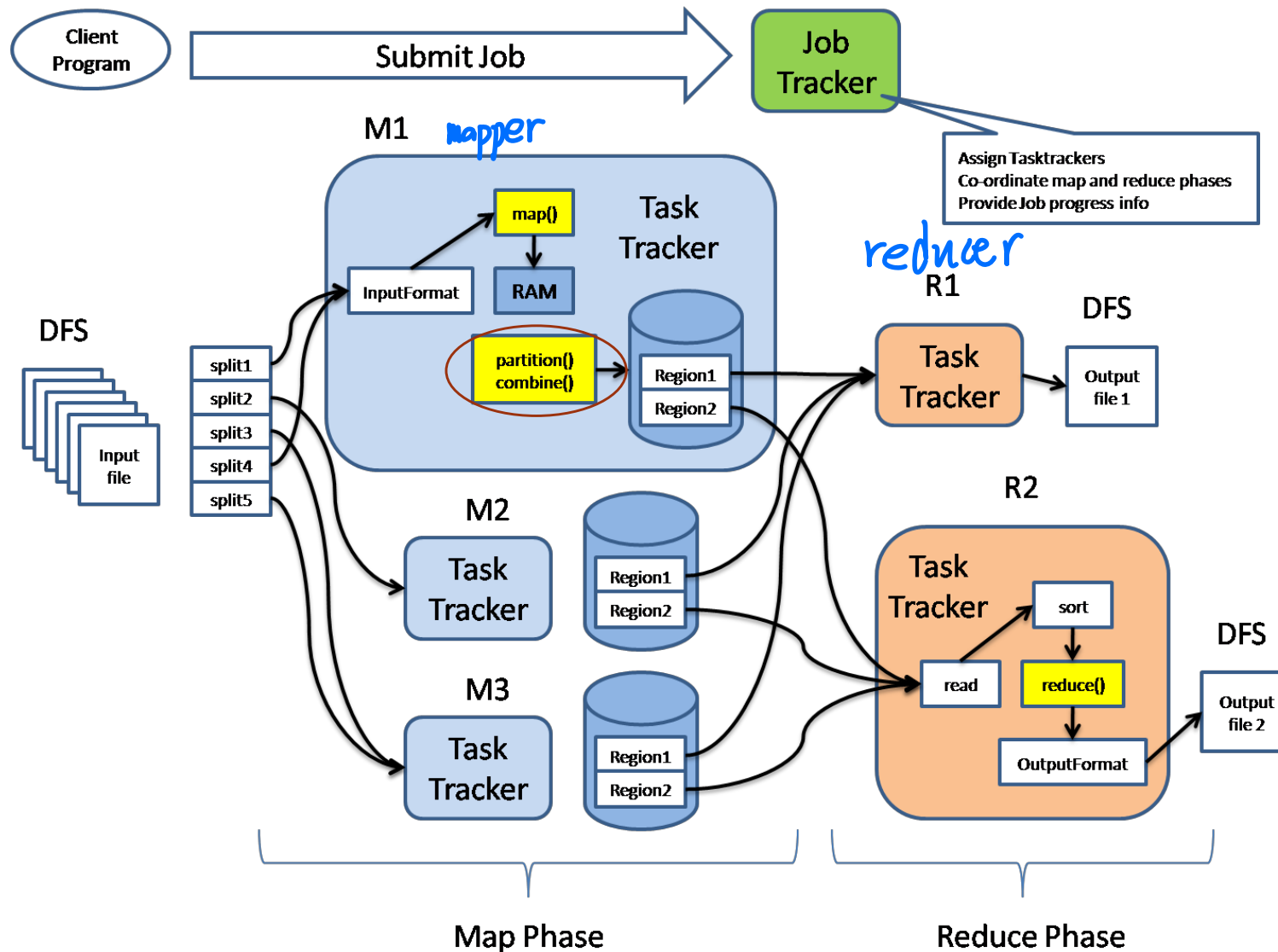
A MapReduce 集群 cluster

- Nodes inside a MapReduce cluster are decomposed as follows:
 - A jobtracker acts as a master node; MapReduce jobs are submitted to it
 - Several tasktrackers run the computation itself, i.e., map and reduce tasks.
 - A given tasktracker may run several tasks in parallel
 - Tasktrackers usually also act as data nodes of a distributed file system (e.g., GFS, HDFS)
- A client node where the application is launched

Processing a MapReduce job

- A MapReduce **job** takes care of the distribution, synchronization and failure handling
 - the input is split into **M** groups; each group is assigned to a **mapper** (assignment is based on the data locality principle)
 - each mapper processes a group and stores the intermediate pairs locally
 - (*shuffle*) intermediate pairs are sorted on their key by the reducer
 - one obtains grouped instances, submitted to the reduce function
 - grouped instances are assigned to **reducers** thanks to a hash function
- **Remark**: the data locality does no longer hold for the reduce phase, since it reads from the mappers

Distributed execution of a MapReduce job



減少傳輸量

Combiners

- A mapper task can produce a large number of pairs with the same key
- They need to be sent over the network to the reducer: **costly**
- It is possible to **combine** these pairs into a single key-value pair
 - Example: (jaguar, 1), (jaguar, 1), (jaguar, 1), (jaguar, 1) $\rightarrow$ (jaguar, 4)
- **Combine**: $\text{list}(v') \rightarrow v''$ function executed (possibly several times) to **combine the values for a given key** on a mapper node
- Easy case: the combiner is the same as the reduce function. Possible when the aggregate function f computed by reduce is **distributive**: $f(k_1, f(k_2, k_3)) = f(k_1, k_2, k_3)$

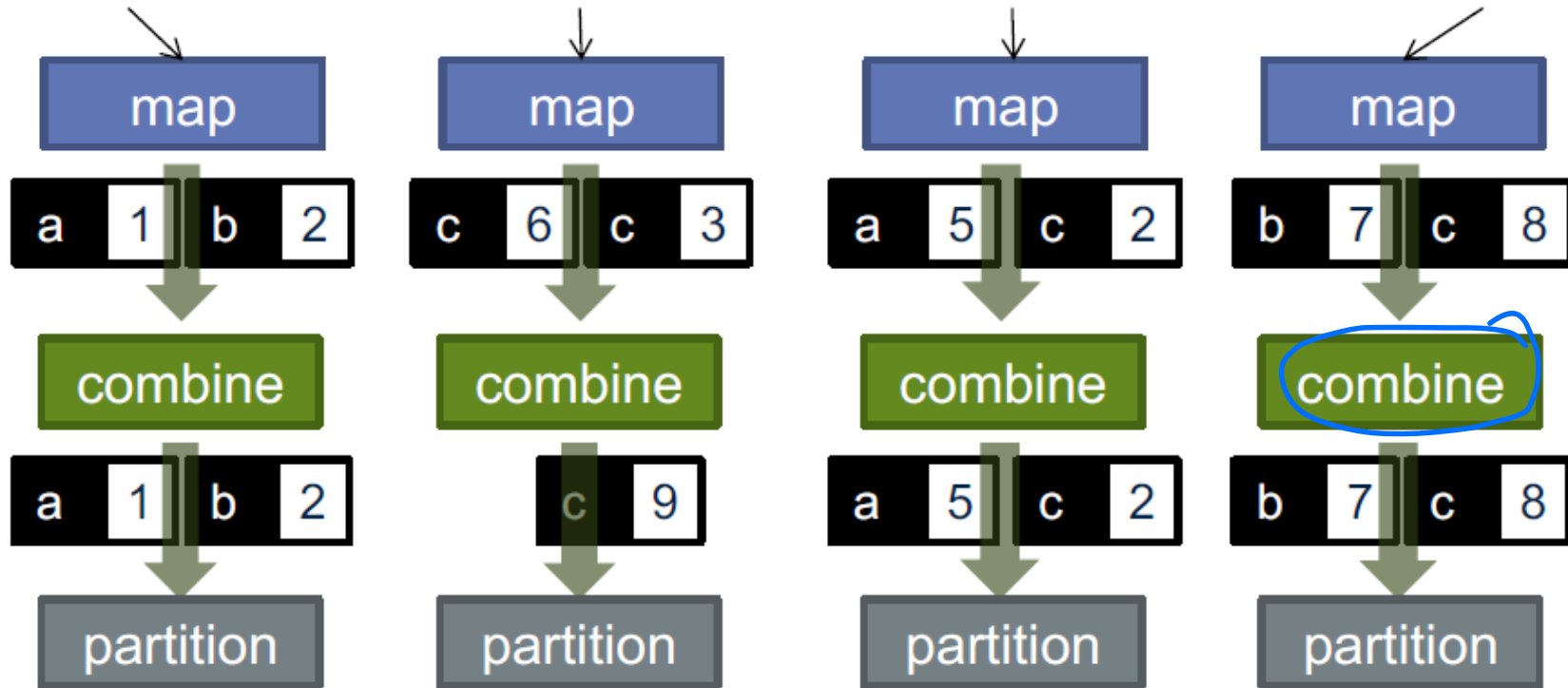
Processing the term count example

- Let the input consist of documents, say, one **million** 100-terms documents of approximately 1 KB each
- The split operation distributes these documents in groups of 64 MBs: each group consist of **64,000** documents. Therefore
 - $M = 1,000,000 / 64,000 \sim 16,000$ groups
- If there are **1,000** mapper node, each node processes on average **16** splits
- *Partition*: If there are **n** reducers, each reducer R_i processes all key-value pairs for terms **t** such that **hash(t) = i**

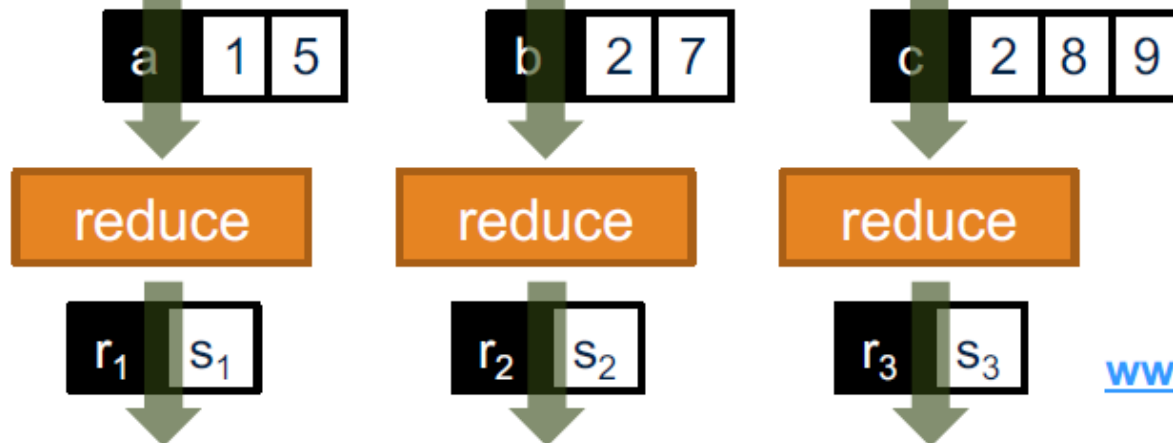
Processing the term count example

- Assume that $\text{hash}(\text{'call'}) = \text{hash}(\text{'mine'}) = \text{hash}(\text{'blog'}) = \mathbf{i} = 100$. We focus on three Mappers M_p , M_q and M_r :
 - $G_p(\mathbf{i}) = (< \dots, (\text{'mine'}, 1), \dots, (\text{'call'}, 1), \dots, (\text{'mine'}, 1), \dots, (\text{'blog'}, 1) >$
 - $G_q(\mathbf{i}) = (< \dots, (\text{'call'}, 1), \dots, (\text{'blog'}, 1), \dots >$
 - $G_r(\mathbf{i}) = (< \dots, (\text{'blog'}, 1), \dots, (\text{'mine'}, 1), \dots, (\text{'blog'}, 1), \dots >$
- Shuffle*: read $G_p(\mathbf{i})$, $G_q(\mathbf{i})$ and $G_r(\mathbf{i})$ from the three Mappers, sort their unified content, and group the pairs with a common key:
 - $\dots, (\text{'blog'}, <1, 1, 1, 1>), \dots, (\text{'call'}, <1, 1>), \dots, (\text{'mine'}, <1, 1, 1>)$
- The reducer R_i receive elements 'blog', 'call', and 'mine' whose hash value is $\mathbf{i}$. The output is ('blog', 4), ('call', 2) and ('mine', 3)

$k_1 \ v_1$ $k_2 \ v_2$ $k_3 \ v_3$ $k_4 \ v_4$ $k_5 \ v_5$ $k_6 \ v_6$ $k_7 \ v_7$ $k_8 \ v_8$



shuffle and sort: aggregate values by keys



Failure management

- In case of failure, because the tasks are distributed over hundreds or thousands of machines, the chances that problems occur somewhere are much larger; starting the job from the beginning is not a valid option.
- The master periodically checks the availability and reachability of the tasktrackers and the progress of map or reduce jobs
 - if a reducer fails, its task is reassigned to another tasktracker; this usually requires the restart of mapper tasks as well (to produce intermediate groups)
 - if a mapper fails, its task is reassigned to another tasktracker
 - if the jobtracker fails, the whole job should be re-initiated

推测 Speculative execution

- The MapReduce jobtracker tries detecting tasks that take longer than usual (e.g., because of hardware problems)
- When detected, such a task is *speculatively* executed on another tasktracker, without killing the existing task
- Eventually, when one of the attempts succeeds, the other one is killed

detect slow task

Using MapReduce for solving a problem

- Prefer:
 - Simple *map* and *reduce* functions
 - Mapper tasks process large data chunks at least the size of distributed filesystem blocks
- A given application may have:
 - A chain of map functions (input processing, filtering, extraction. . .)
 - A sequence of several map-reduce jobs
 - No reduce task when everything can be expressed in the map (zero reducers, or the identity reducer function)
- Not the right tool for everything (why we have Spark?)

Homework

- <http://web.mit.edu/6.033/2017/wwwdocs/assignments/handson-mapreduce.html>
 - Run the script: `python mapreduce.py kjv12.txt`
- Writing a MapReduce to compute tfidf representations using the data set [KOS blog entries](https://archive.ics.uci.edu/ml/datasets/Bag+of+Words) in <https://archive.ics.uci.edu/ml/datasets/Bag+of+Words>
- Google Colab: Hadoop and Spark

Outline

- The MapReduce framework

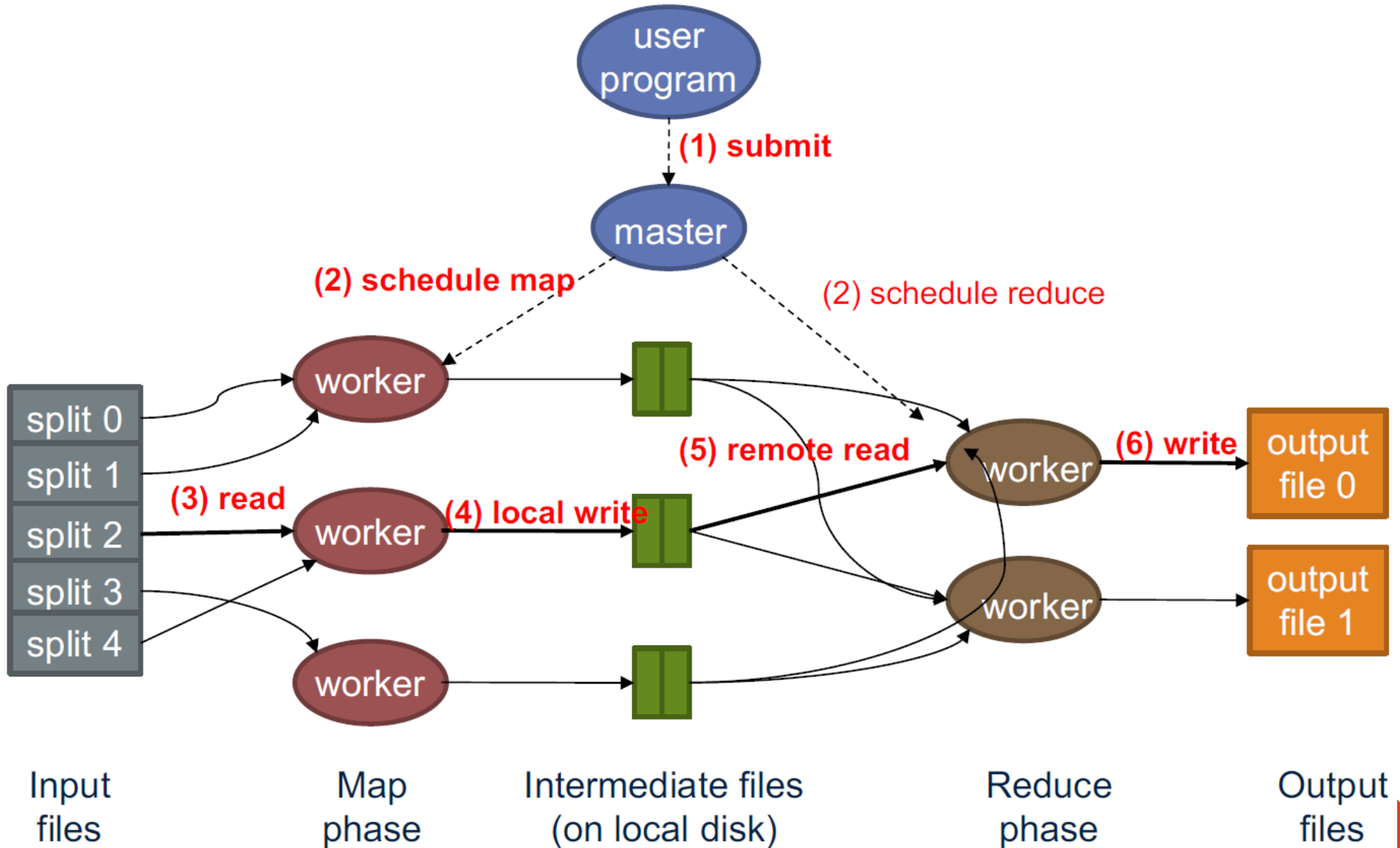
- MapReduce
- HDFS
- Apache Hadoop
- Apache Spark

history understands for a new distributed system.

- Applications:

- PageRank
- Similarity join

Overview

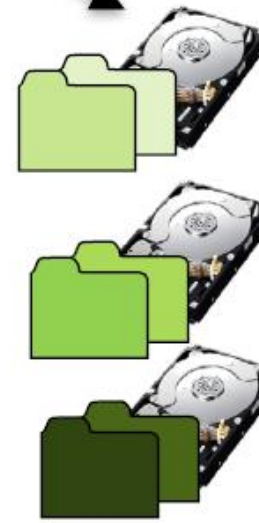


How worker get data?

client machine



cluster



file server farm
(NAS, SAN,..)



What's the problem here?

Distributed file system

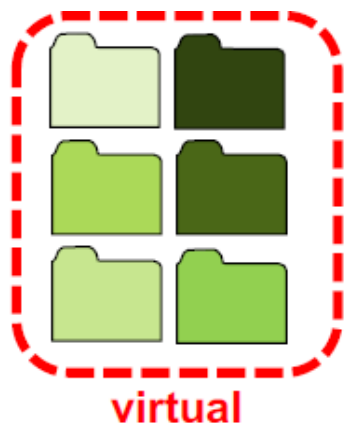
- We move workers to the data

- Store data on the local disks of nodes in the cluster
- Start up the workers on the node that has the data local

*move computation
to where the
data is.*

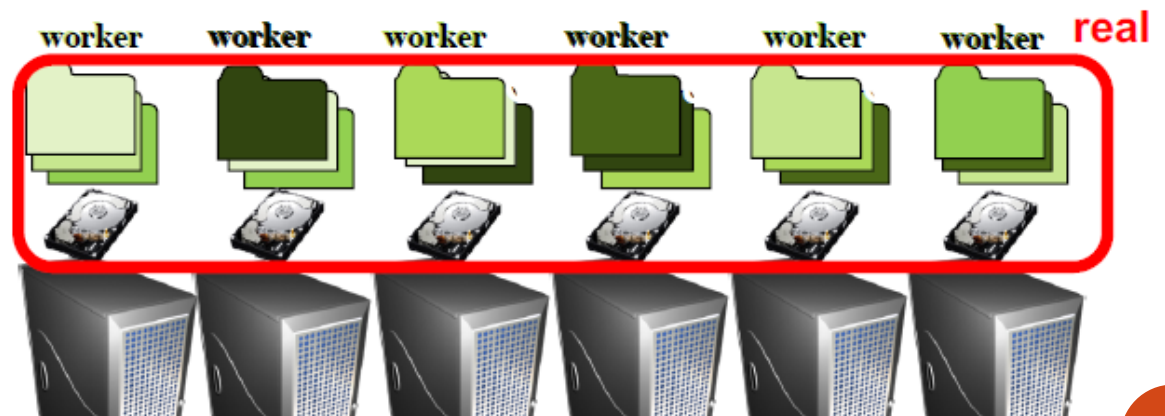
- Why?

- Avoid network traffic if possible
- Not enough RAM to hold all the data in memory



MapReduce Job →

**HDFS (GFS)
Distributed
File-system**



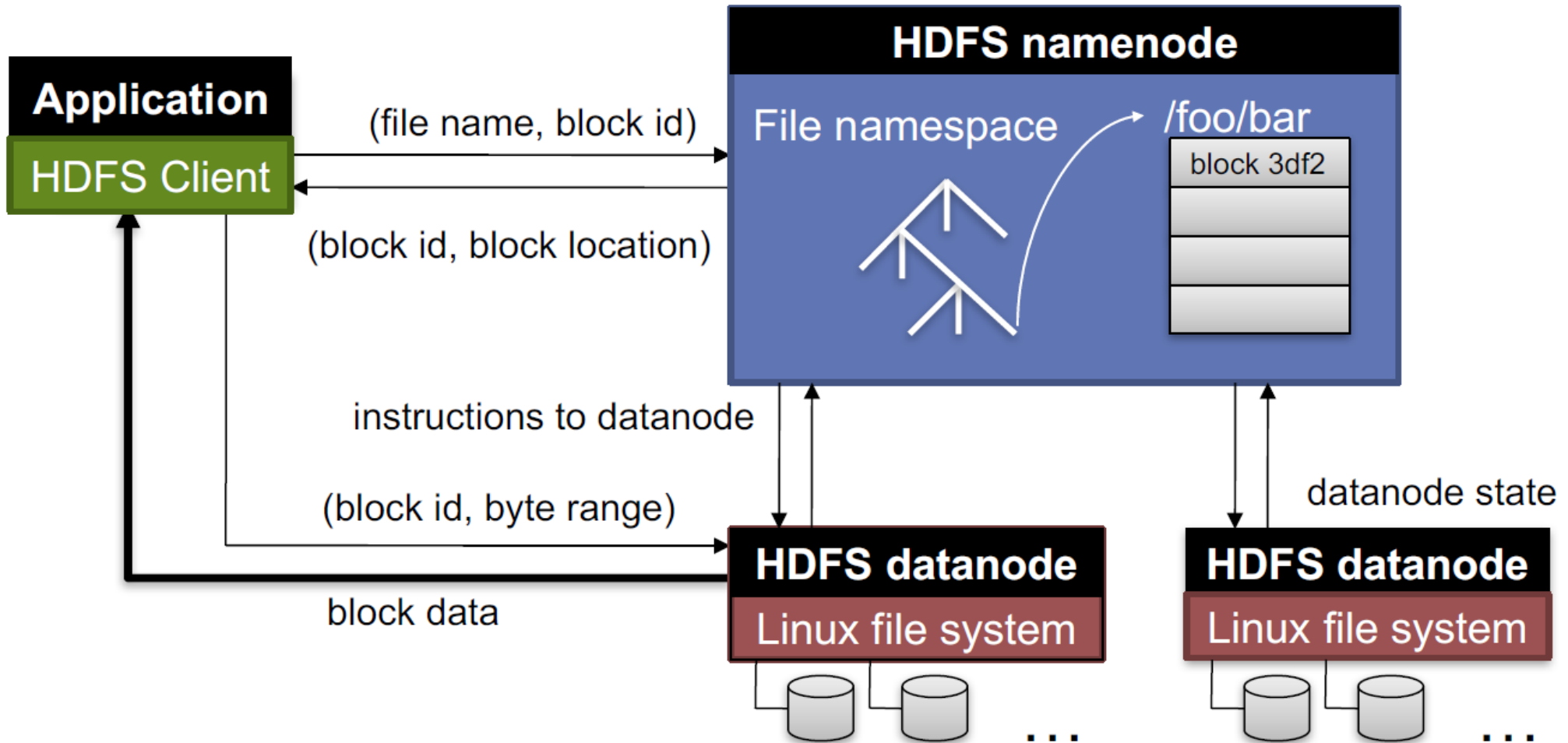
GFS vs HDFS

- Google file system (GFS) properties:
 - Files stored as chunks (each 64 MB)
 - Each chunk replicated across 3+ chunkservers for reliability
 - Single master to coordinate access and keep metadata
 - No data caching due to large datasets, streaming reads
 - Simplify the API
- Hadoop distributed file system (HDFS):
 - Same basic properties as GFS

GFS vs. HDFS

- Terminology differences:
 - GFS master = Hadoop namenode
 - GFS chunkservers = Hadoop datanodes
- Differences:
 - Different consistency model for file appends
 - Implementation & performance

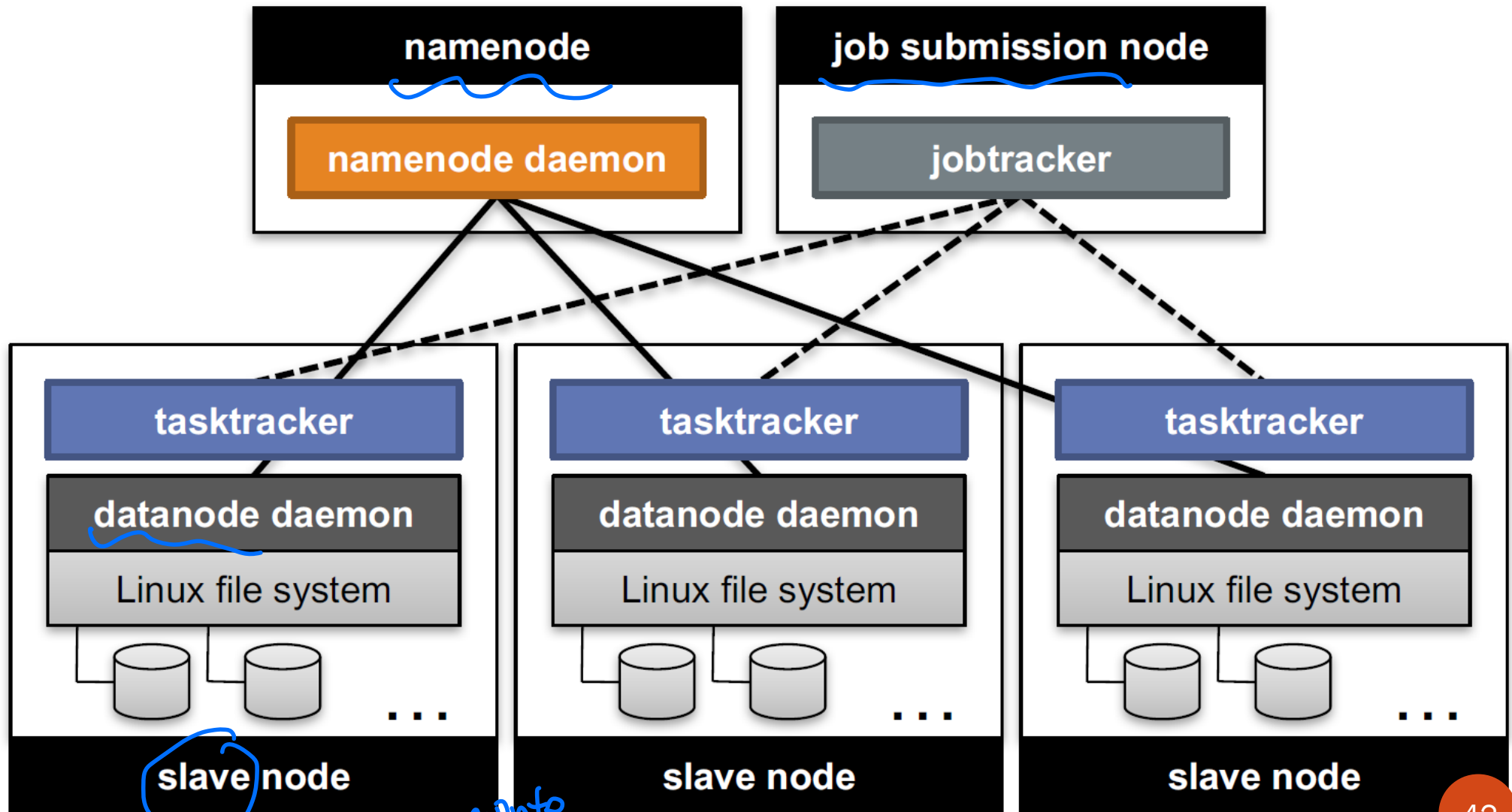
HDFS architecture



Namenode responsibility

- Managing the file system namespace:
 - Hold file/directory structure, metadata, file-to-block mapping, access permissions, etc
- Coordinating file operations:
 - Directs clients to datanodes for reads and writes
 - No data is moved through the namenode
- Maintain overall health:
 - Periodic communication with the datanodes
 - Block re-replication and rebalancing
 - Garbage collection

HDFS overview



奴隶 store, compute

Outline

- The MapReduce framework
 - MapReduce
 - HDFS
 - Apache Hadoop
 - Apache Spark
- Applications:
 - PageRank
 - Similarity join

Hadoop

- Open-source software, Java-based, managed by the Apache foundation, for large-scale distributed storage and computing
- Open-source implementation of GFS and Google's MapReduce
- Yahoo!: a main contributor of the development of Hadoop

Hadoop ecosystem

- Basic services:
 - HDFS: Open-source GFS clone originally funded by Yahoo
 - MapReduce: Open-source MapReduce implementation (Java, Python)
 - Yarn: Resource manager to share clusters between MapReduce and other tools
 - Spark: new in-memory MapReduce++ based on Scala (avoid HDFS writes)
- Data querying:
 - Hive, Pig, Impala: SQL system running on top of Hadoop
 - SparkSQL: SQL system running on top of Spark
- Graph processing:
 - Giraph: Pregel clone on Hadoop
 - GraphX: graph analysis library of Spark
- Machine learning:
 - Mlib: Spark based library of machine learning algorithms

Hadoop programming interfaces

- Different APIs to write Hadoop programs:
 - A rich **Java** API (main way to write Hadoop programs)
 - A **C++** API (Hadoop Pipes)
 - A **streaming** API that can be used to write *map* and *reduce* functions in any programming language (using standard inputs and outputs)
 - With a **higher-language level** (e.g., Pig, Hive)
- Advanced features only available in the Java API

Java map for the term count example

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
    ) throws IOException, InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

Java reduce for the term count example

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
    ) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Java driver for the term count example

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```



Latest news

Release 3.4.1 available

2024 Oct 18

This is a release of Apache Hadoop 3.4.1 line.

Users of Apache Hadoop 3.4.0 and earlier should upgrade to this release.

All users are encouraged to read the [overview of major changes](#) since release 3.4.0.

We have also introduced a lean tar which is a small tar file that does not contain the AWS SDK because the size of AWS SDK is itself 500 MB. This can ease usage for non AWS users. Even AWS users can add this jar explicitly if desired.

For details of bug fixes, improvements, and other enhancements since the previous 3.4.0 release, please check [release notes](#) and [changelog](#).

Modules

The project includes these modules:

- **Hadoop Common:** The common utilities that support the other Hadoop modules.
- **Hadoop Distributed File System (HDFS™):** A distributed file system that provides high-throughput access to application data.
- **Hadoop YARN:** A framework for job scheduling and cluster resource management.
- **Hadoop MapReduce:** A YARN-based system for parallel processing of large data sets.

Who Uses Hadoop?

A wide variety of companies and organizations use Hadoop for both research and production. Users are encouraged to add themselves to the Hadoop [PoweredBy wiki page](#).

What is Apache Spark?

- Fast and expressive cluster computing system inter-operable with Apache Hadoop
- Improve efficiency (up to **100x** faster) through:
 - In-memory computing primitives
 - Optimized based on computation graphs
- Improve usability (**5x** less code) through:
 - Rich APIs in Scala, Python, Java
 - Interactive shell

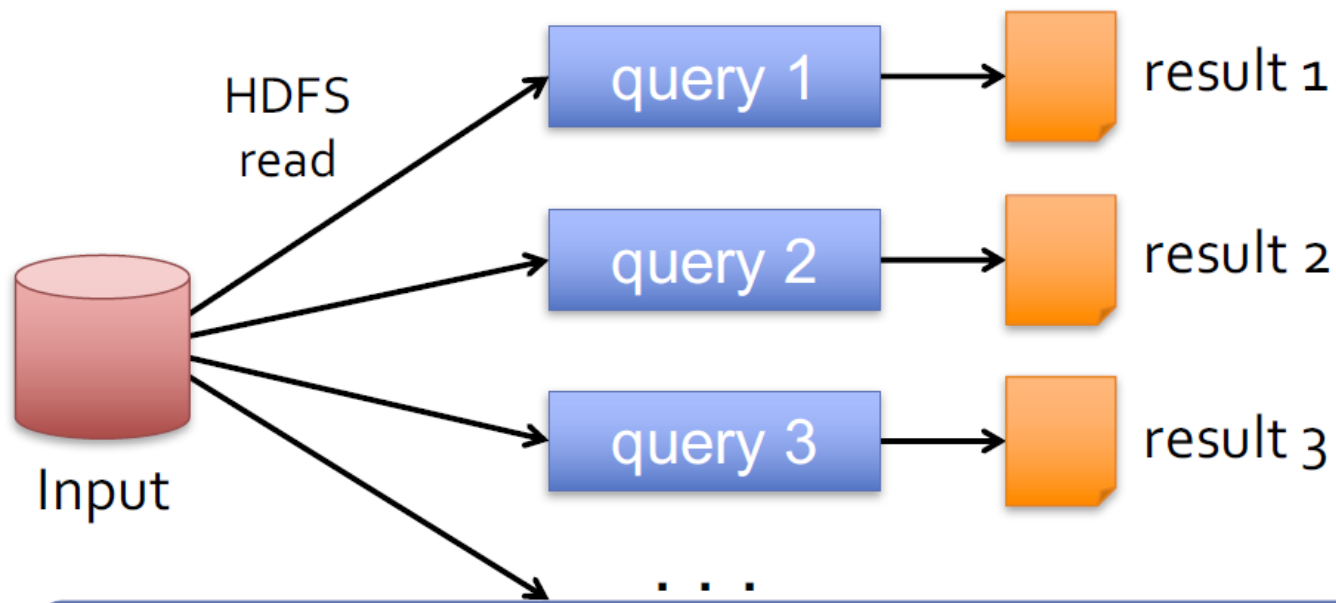
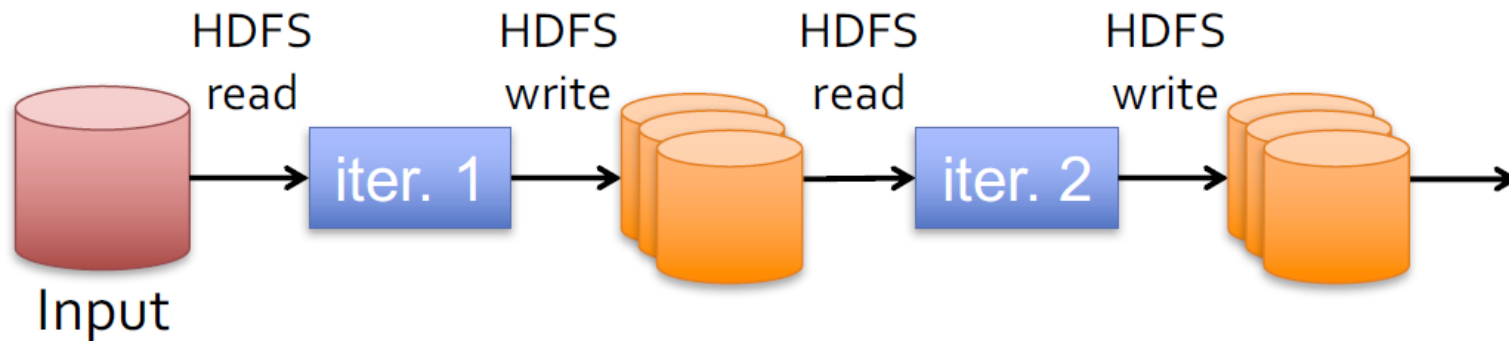
Why a new programming model?

disk

- MapReduce greatly simplified big data analysis
- But as soon as it got popular, users wanted more:
 1. More **complex**, multi-pass analysis (e.g. ML, graph)
 2. More **interactive** ad-hoc queries
 3. More **real-time** stream processing
- All 3 need faster **data sharing** across parallel jobs

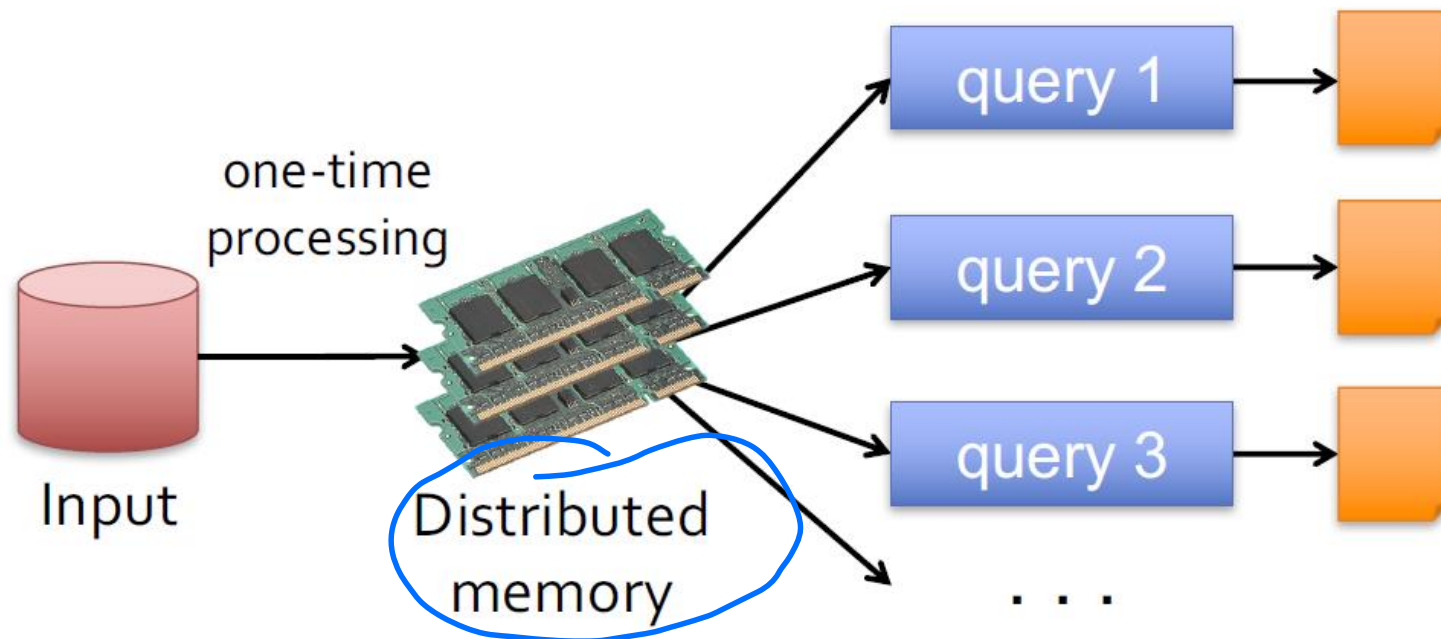
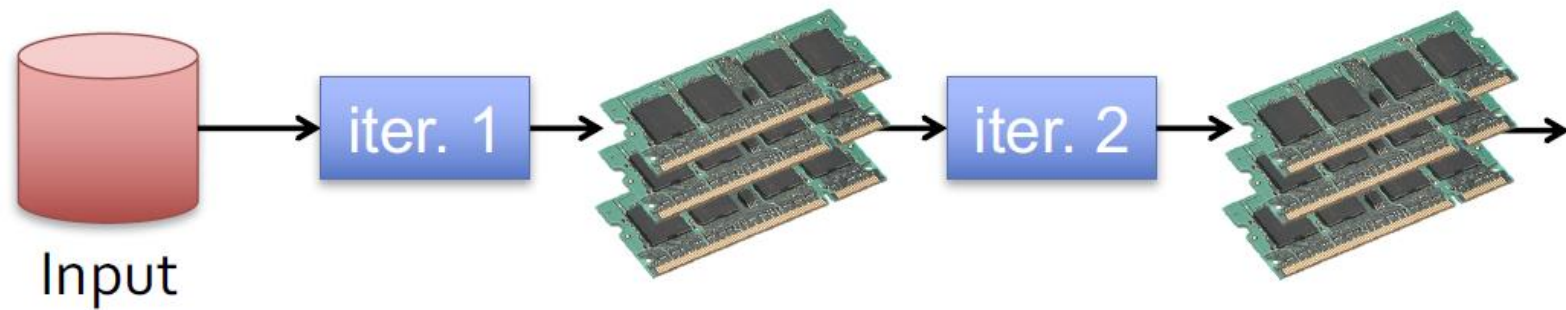
Spark
#F107

Data sharing in MapReduce



Slow due to replication, serialization, and disk IO

Data sharing in Spark

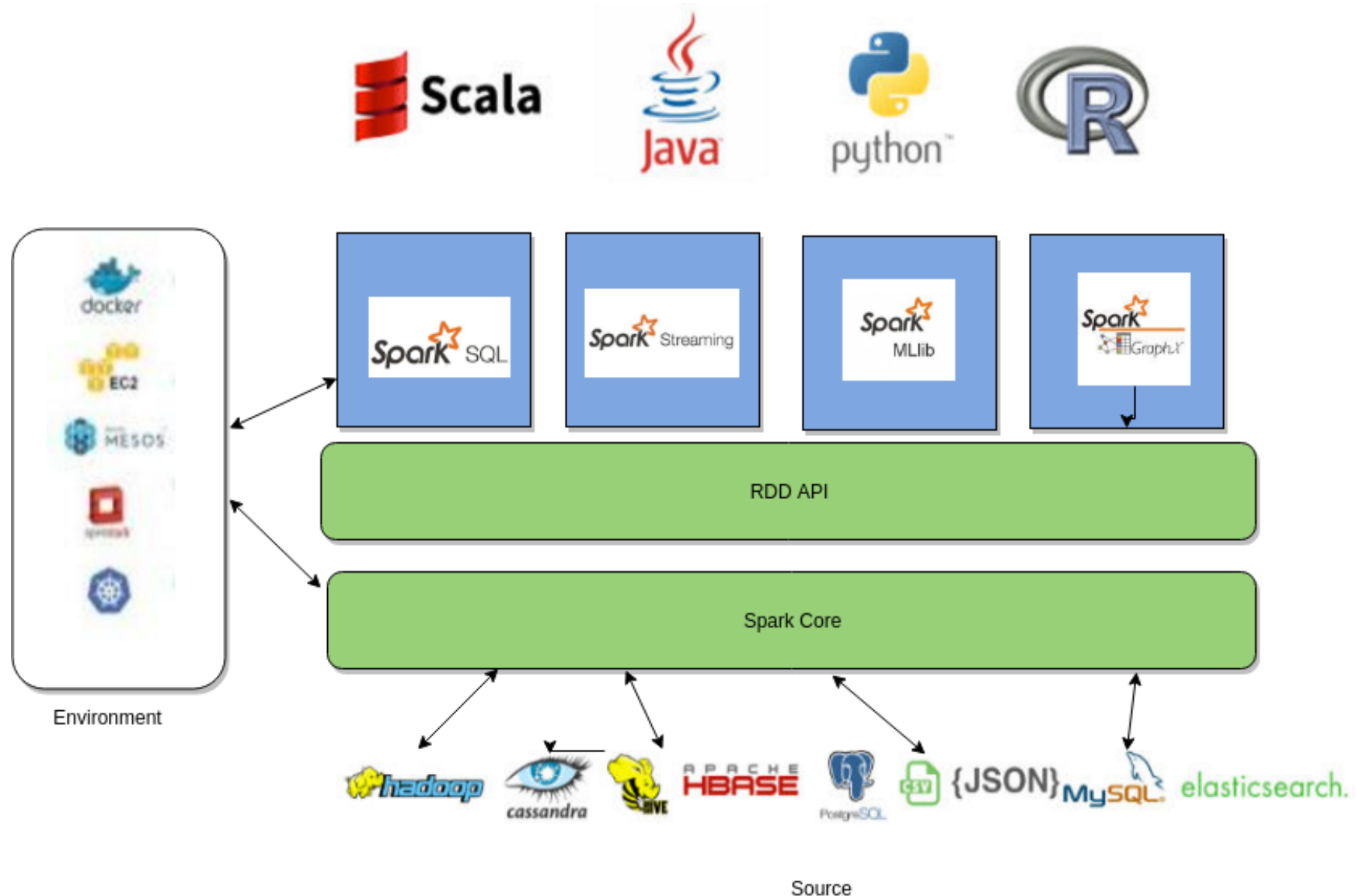


~10 × faster than network and disk

Spark programming model

- Key idea: ^{弹性.}resilient distributed datasets (RDDs)
 - Distributed collections of objects that can be cached ^{缓存.} in memory across the cluster
 - Manipulated through parallel operators
 - Automatically recomputed on failure
- Programming interface:
 - Functional APIs in Scala, Java, Python
 - Interactive use from Scala shell ^{交互}

Apache Spark components



Hadoop vs. Spark

	Hadoop	Spark
Category	Basic data processing engine	Data analytics engine
Usage	Batch processing with a huge volume of data	Process real-time data, from real-time events like Twitter, Facebook
Latency	High latency computing	Low latency computing
Data	Process data in batch mode <i>chunk</i>	Can process interactively
Ease of use	Hadoop's MapReduce model is complex, need to handle low-level APIs	Easier to use, abstraction enables a user to process data using high-level operators
Scheduler	<u>External</u> job scheduler is required <i>Yung</i>	<u>In-memory</u> computation, <u>no external scheduler required</u>
Security	Highly secure	<u>Less</u> secure as compare to Hadoop
Cost	Less costly since MapReduce model provide a <u>cheaper</u> strategy	<u>Costlier</u> than Hadoop since it has an <u>in-memory solution</u>